



SQL INTERVIEW GUIDE

30 Real Interview Questions with Scenarios

Free resource by **GRASSP Acad**

Useful for **Data Analyst, Data Engineer, Data Scientist, and AI/ML Engineer** interviews — every question below is framed as a real business scenario, not a textbook definition.

Contents

- 1.** WHERE vs HAVING
- 2.** COUNT(*) vs COUNT(column)
- 3.** GROUP BY Rules
- 4.** JOIN Types (INNER vs LEFT)
- 5.** Anti-Join (Rows in Table A with No Match in Table B)
- 6.** Primary Key vs Foreign Key
- 7.** Column Aliases in WHERE
- 8.** COALESCE
- 9.** SUM with NULL values
- 10.** ORDER BY on a Column Not in SELECT
- 11.** CASE With No ELSE
- 12.** CASE Inside an Aggregate Function
- 13.** Pivoting Rows to Columns With CASE
- 14.** IN vs NOT IN, and NULL Handling
- 15.** INNER JOIN vs LEFT JOIN (Fundamentals)
- 16.** JOINing on a Column With NULL Values
- 17.** RANK() vs DENSE_RANK()
- 18.** LAG() vs LEAD()
- 19.** CTE vs Subquery
- 20.** DATE_TRUNC vs EXTRACT vs TO_CHAR
- 21.** STRING_AGG()
- 22.** FOREIGN KEY Violation on INSERT
- 23.** Percentage of Total (Window Function)
- 24.** Indexes
- 25.** Views
- 26.** Normalization
- 27.** DELETE/UPDATE Without WHERE
- 28.** DELETE vs TRUNCATE vs DROP
- 29.** Replacing NULLs — COALESCE vs CASE
- 30.** Finding Duplicates — Two Ways

1. WHERE vs HAVING

Scenario: Find every store where total Chicken Burger sales crossed \$5,000 last month.

✗ Wrong:

```
SELECT store_id, SUM(sales) AS total_sales
FROM orders
WHERE item = 'Chicken Burger' AND SUM(sales) > 5000
GROUP BY store_id;
```

✓ Correct:

```
SELECT store_id, SUM(sales) AS total_sales
FROM orders
WHERE item = 'Chicken Burger'
GROUP BY store_id
HAVING SUM(sales) > 5000;
```

What it tests:

- WHERE filters individual rows before grouping.
- GROUP BY creates the groups.
- SUM() is calculated per group.
- HAVING filters the aggregated results.
- Since SUM(sales) doesn't exist until after GROUP BY, it can't be used in WHERE.

Interview Tip:

- *Say it in terms of execution order — "WHERE filters raw rows first, then GROUP BY aggregates, then HAVING filters the result." That shows you understand execution order, not just syntax.*

2. COUNT(*) vs COUNT(column)

Scenario: How many orders does SFF-BLR have, and how many of those have a non-null order_date?

```
SELECT
  COUNT(*) AS total_orders,
  COUNT(order_date) AS orders_with_order_date
FROM orders
WHERE store_id = 'SFF-BLR';
```

What it tests:

- COUNT(*) counts all rows regardless of NULLs.
- COUNT(column) counts only non-NULL values in that column.

Interview Tip:

- *Call out the NULL-handling explicitly — "COUNT(column) silently skips NULLs, that's a common source of mismatched counts in reports."*

3. GROUP BY Rules

Scenario: Calculate the total Fries revenue generated by each store, for every month.

✗ Wrong:

```
SELECT store_id, order_date, SUM(sales) AS total_sales
FROM orders
WHERE item = 'Fries'
GROUP BY store_id;
```

✓ Correct:

```
SELECT store_id, DATE_TRUNC('month', order_date) AS month, SUM(sales) AS total_sales
FROM orders
WHERE item = 'Fries'
GROUP BY store_id, DATE_TRUNC('month', order_date);
```

What it tests:

- Every non-aggregated column in SELECT — including expressions like DATE_TRUNC — must appear in GROUP BY.
- DATE_TRUNC('month', order_date) rounds every date down to the 1st of its month, keeping the year intact, so days within the same month and year collapse into one group.

Interview Tip:

- *Explain it as an expression, not just a column rule — "It's a computed expression and not aggregated, so it still has to be in GROUP BY — same rule, just applied to an expression instead of a plain column."*

4. JOIN Types (INNER vs LEFT)

Scenario: List every SFF-NYC customer and their order count — including customers who've never ordered.

```
-- INNER JOIN: only customers with at least one order
SELECT c.customer_id, COUNT(o.order_id) AS order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE c.store_id = 'SFF-NYC'
GROUP BY c.customer_id;
```

```
-- LEFT JOIN: all customers, orders or not
SELECT c.customer_id, COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.store_id = 'SFF-NYC'
```

```
GROUP BY c.customer_id;
```

What it tests:

- INNER JOIN only keeps matching rows between two tables.
- LEFT JOIN keeps every row from the left table, with NULLs (→ 0 via COUNT) for non-matches.

Interview Tip:

- *Frame it around business risk — "Using INNER JOIN here would exclude never-ordered customers from a churn report. LEFT JOIN includes them, which is usually what the business actually wants."*

5. Anti-Join (Rows in Table A with No Match in Table B)

Scenario: Find customers who have never placed an order.

```
-- LEFT JOIN + IS NULL
SELECT c.customer_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL;

-- Alternative: EXCEPT
SELECT customer_id FROM customers
EXCEPT
SELECT customer_id FROM orders;

-- Alternative: NOT EXISTS
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

What it tests:

- LEFT JOIN + IS NULL on the right table's key is the classic anti-join pattern — non-matches produce NULLs on the joined side.
- NOT EXISTS uses a correlated subquery and is often the better performer on large tables, since the database can stop searching as soon as it finds one match.

Interview Tip:

- *Mention all three, and if asked "why SELECT 1 inside NOT EXISTS" — "with EXISTS/NOT EXISTS, the optimizer only checks for row existence, so the selected column is irrelevant. SELECT 1 is the convention to make that explicit."*

6. Primary Key vs Foreign Key

Scenario: stores.store_id is the primary key — every store has one unique ID (SFF-BLR, SFF-NYC, SFF-LDN, SFF-DXB). orders.store_id is a foreign key referencing it.

```
INSERT INTO orders (order_id, store_id, item, sales)
VALUES (1001, 'SFF-TOKYO', 'Chicken Burger', 250);
-- Fails - 'SFF-TOKYO' doesn't exist in the stores table
```

What it tests:

- A primary key uniquely identifies each row in its own table (never duplicated, never null).
- A foreign key is a column in a different table that must match a value that already exists in the primary key it references.
- This is called referential integrity.

Interview Tip:

- *"Primary key = uniquely identifies a row in its own table. Foreign key = a column in another table that must point to a valid primary key value. Together they enforce that relationships between tables stay valid."*

7. Column Aliases in WHERE

Scenario: You calculate discounted_order_amount in SELECT. Can you filter on it using WHERE?

✗ Wrong:

```
SELECT order_id, order_amount, order_amount * 0.5 AS discounted_order_amount
FROM orders
WHERE discounted_order_amount > 100;
```

✓ Correct:

```
SELECT order_id, order_amount, order_amount * 0.5 AS discounted_order_amount
FROM orders
WHERE order_amount * 0.5 > 100;
```

What it tests:

- WHERE is evaluated before SELECT, so the alias doesn't exist yet at that stage.

Interview Tip:

- *Recite the SQL logical execution order — FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. WHERE runs before SELECT assigns the alias.*

8. COALESCE

Scenario: customers.email and customers.phone sometimes have NULLs. Display "NA" instead of a blank value for each, independently.

```
SELECT
    customer_id,
    COALESCE(email, 'NA') AS email,
    COALESCE(phone, 'NA') AS phone
FROM customers;
```

What it tests:

-) returns the first non-NULL value from its list.
- Each column is evaluated independently — a customer missing only their phone still shows their real email.

Interview Tip:

- *"It returns the first non-NULL value from a list of arguments — commonly used to substitute a default value wherever a column might be NULL. You can apply it separately to multiple columns in the same query, each with its own fallback."*

9. SUM with NULL values

Scenario: orders.order_amount has some NULL entries. What does SUM(order_amount) actually do with them?

```
SELECT store_id, SUM(order_amount) AS total_sales
FROM orders
WHERE store_id = 'SFF-BLR'
GROUP BY store_id;
```

What it tests:

- SUM() (like AVG, MAX, MIN, COUNT(column)) automatically ignores NULLs — it does not treat them as 0.

Interview Tip:

- *A sharp follow-up: "What if you wanted NULLs treated as 0?" Answer: wrap the column — SUM(COALESCE(order_amount, 0)) — the total is mathematically identical since NULLs are skipped either way, but COALESCE matters when you need the raw per-row value to be 0 for other calculations.*

10. ORDER BY on a Column Not in SELECT

Scenario: List item names sold at SFF-LDN, sorted by order_amount — without displaying order_amount itself.

```
SELECT item
```

```
FROM orders
WHERE store_id = 'SFF-LDN'
ORDER BY order_amount DESC;
```

What it tests:

- ORDER BY runs after SELECT but can still reference columns from the underlying tables even if they're not in the SELECT list, because it operates on the query's source data.
- The exception: with SELECT DISTINCT, the sort column must be part of the distinct result set.

Interview Tip:

- *"ORDER BY can reference any column from the FROM/JOIN sources, regardless of what's in SELECT — the exception is SELECT DISTINCT, where SQL needs the sort column to already be part of the distinct result set."*

11. CASE With No ELSE

Scenario: You categorize orders by size using CASE, but forgot an ELSE branch.

✗ Missing ELSE:

```
SELECT order_id, order_amount,
       CASE
         WHEN order_amount > 500 THEN 'High'
         WHEN order_amount > 100 THEN 'Medium'
       END AS sales_tier
FROM orders;
```

✓ With ELSE:

```
SELECT order_id, order_amount,
       CASE
         WHEN order_amount > 500 THEN 'High'
         WHEN order_amount > 100 THEN 'Medium'
         ELSE 'Low'
       END AS sales_tier
FROM orders;
```

What it tests:

- If no WHEN matches and there's no ELSE, CASE returns NULL — no error, no warning.
- That NULL can silently sneak into downstream aggregations or reports.

Interview Tip:

- *"A CASE without ELSE doesn't fail loudly — it returns NULL for unmatched rows. That's more dangerous than an error because it passes silently into a report and looks like a data quality issue instead of a logic gap. Good practice to always add ELSE."*

12. CASE Inside an Aggregate Function

Scenario: One row per store, with separate columns for total Chicken Burger sales and total Fries sales — without two separate queries.

```
SELECT
    store_id,
    SUM(CASE WHEN item = 'Chicken Burger' THEN sales ELSE 0 END) AS
chicken_burger_sales,
    SUM(CASE WHEN item = 'Fries' THEN sales ELSE 0 END) AS fries_sales
FROM orders
GROUP BY store_id;
```

What it tests:

- CASE can sit inside SUM, COUNT, AVG, or any aggregate.
- It runs first, per row, converting the value into either its real amount or 0.
- This pattern is called conditional aggregation.

Interview Tip:

- *"CASE evaluates per row before the aggregate runs, so I can conditionally include or zero-out a value per row, then let SUM/COUNT aggregate as normal — it's how you get multiple conditional metrics out of one query."*

13. Pivoting Rows to Columns With CASE

Scenario: One row per store, with each item's sales as its own column — Chicken Burger, Veg Pizza, Cold Coffee, Fries side by side.

```
SELECT
    store_id,
    SUM(CASE WHEN item = 'Chicken Burger' THEN sales ELSE 0 END) AS chicken_burger,
    SUM(CASE WHEN item = 'Veg Pizza' THEN sales ELSE 0 END) AS veg_pizza,
    SUM(CASE WHEN item = 'Cold Coffee' THEN sales ELSE 0 END) AS cold_coffee,
    SUM(CASE WHEN item = 'Fries' THEN sales ELSE 0 END) AS fries
FROM orders
GROUP BY store_id;
```

What it tests:

- This is conditional aggregation extended across every value — a manual way of pivoting data from long format into wide format.
- It's what PIVOT/crosstab does natively in databases that support it, but CASE + SUM works in every SQL engine.

Interview Tip:

- *"This is a manual pivot — CASE + an aggregate function turns row-based category values into columns. It's the go-to technique when the database doesn't support PIVOT natively."*

14. IN vs NOT IN, and NULL Handling

Scenario: Find customers who have never placed an order, using NOT IN against the orders table.

```
-- IN - works as expected
SELECT customer_id
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- NOT IN - dangerous with NULLs
SELECT customer_id
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
-- If orders.customer_id contains even one NULL, this returns ZERO rows

-- Safer alternative: NOT EXISTS
SELECT c.customer_id
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

What it tests:

- IN checks if a value exists in a list — NULLs in the list don't usually cause problems.
- NOT IN checks if a value is not in the list — but if the list contains even one NULL, comparisons with NULL become UNKNOWN, so the condition never evaluates to TRUE, and the query can silently return zero rows.

Interview Tip:

- *"NOT IN is unsafe if the subquery can return NULL, since NULL comparisons evaluate to UNKNOWN, not FALSE. I use NOT EXISTS instead, because it checks row existence directly and isn't affected by NULLs."*

15. INNER JOIN vs LEFT JOIN (Fundamentals)

Scenario: List SFF-BLR customers and their orders — with and without requiring an order to exist.

```
-- INNER JOIN: only customers with at least one order
SELECT c.customer_id, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
WHERE c.store_id = 'SFF-BLR';

-- LEFT JOIN: all customers, orders or not
SELECT c.customer_id, o.order_id
FROM customers c
```

```
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.store_id = 'SFF-BLR';
```

What it tests:

- INNER JOIN returns only rows with a match in both tables.
- LEFT JOIN returns all rows from the left table regardless of a match, filling unmatched right-side columns with NULL.

Interview Tip:

- *"INNER JOIN requires a match on both sides. LEFT JOIN guarantees every left-table row appears at least once, filling unmatched right-side columns with NULL."*

16. JOINing on a Column With NULL Values

Scenario: Due to a data migration issue, both customers.customer_id and orders.customer_id contain some NULLs. Will rows where both are NULL match?

```
-- INNER JOIN
SELECT c.customer_id, c.customer_name, o.order_id
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
-- Rows where customer_id is NULL in either table are never matched — even if both are
NULL

-- LEFT JOIN
SELECT c.customer_id, c.customer_name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
-- All customers returned; NULL customer_id rows still appear, with order_id as NULL
```

What it tests:

- A JOIN only matches rows when the join condition evaluates to TRUE.
- NULL = NULL evaluates to UNKNOWN, not TRUE — so SQL never considers those rows a match, regardless of JOIN type.

Interview Tip:

- *"NULL join keys never match — even when both tables contain NULL — because NULL = NULL evaluates to UNKNOWN, not TRUE. INNER JOIN excludes those rows; LEFT JOIN keeps all rows from the left table and returns NULL for the unmatched columns."*

17. RANK() vs DENSE_RANK()

Scenario: Reward the top 3 highest-value orders in each store — independently per store.

```
WITH ranked_orders AS (
  SELECT order_id, customer_id, store_id, order_amount,
```

```

        RANK() OVER (PARTITION BY store_id ORDER BY order_amount DESC) AS
rank_in_store,
        DENSE_RANK() OVER (PARTITION BY store_id ORDER BY order_amount DESC) AS
dense_rank_in_store
    FROM orders
    WHERE status = 'Completed'
)
SELECT *
FROM ranked_orders
WHERE dense_rank_in_store <= 3;

```

What it tests:

- PARTITION BY store_id resets the ranking for each store independently.
- With ties: RANK() skips the next rank (1,2,2,4), DENSE_RANK() does not (1,2,2,3).

Interview Tip:

- *"PARTITION BY creates independent rankings for each store. If there are ties, RANK() skips rank numbers, whereas DENSE_RANK() assigns consecutive ranks. Use DENSE_RANK() when the business wants the top N distinct ranks."*

18. LAG() vs LEAD()

Scenario: A store manager wants to compare each order's amount with the order placed right before and right after it — within the same store.

```

SELECT order_id, store_id, order_date, total_amount,
       LAG(total_amount) OVER (PARTITION BY store_id ORDER BY order_date) AS
prev_order_amount,
       LEAD(total_amount) OVER (PARTITION BY store_id ORDER BY order_date) AS
next_order_amount
FROM orders
WHERE status = 'Completed';

```

What it tests:

- PARTITION BY store_id keeps the comparison within each store.
- LAG() pulls the previous row's value within that partition; LEAD() pulls the next.
- Both return NULL at the edges (first/last row in each store).

Interview Tip:

- *"We partition by store_id so LAG and LEAD only compare orders within the same store, not across the whole table. LAG gives me the previous order's value, LEAD gives me the next — useful for spotting jumps or drops order-to-order."*

19. CTE vs Subquery

Scenario: First calculate each store's monthly sales, then rank stores within each month — a two-step problem.

```
WITH monthly_orders AS (
    SELECT store_id, DATE_TRUNC('month', order_date) AS month,
           SUM(order_amount) AS total_order_amount
    FROM orders
    GROUP BY store_id, DATE_TRUNC('month', order_date)
)
SELECT store_id, month, total_order_amount,
       RANK() OVER (PARTITION BY month ORDER BY total_order_amount DESC) AS store_rank
FROM monthly_orders;
```

What it tests:

- A CTE (Common Table Expression) is a temporary named result set created with WITH.
- Use one when you have multiple steps, want to make complex SQL easier to read and maintain, or want to reuse an intermediate result within the same query.

Interview Tip:

- *"I prefer CTEs over nested subqueries when a query has multiple steps because they improve readability, make debugging easier, and simplify maintenance."*

20. DATE_TRUNC vs EXTRACT vs TO_CHAR

Scenario: From the same order_date column, you need three different outputs: group by month while keeping the year, get just the month number for a report, and display the month in a readable format for a dashboard.

```
SELECT
    order_id,
    order_date,
    DATE_TRUNC('month', order_date) AS month_start,
    EXTRACT(MONTH FROM order_date) AS month_number,
    TO_CHAR(order_date, 'Mon YYYY') AS display_month
FROM orders;
```

What it tests:

- DATE_TRUNC returns a date rounded to the given precision — used for grouping.
- EXTRACT returns a single numeric component (drops the year).
- TO_CHAR converts a date into formatted text for display.

Interview Tip:

- *"DATE_TRUNC is for grouping dates, EXTRACT is for retrieving a specific date component, and TO_CHAR is for formatting dates as readable text."*

21. STRING_AGG()

Scenario: List all product names under each category as a single comma-separated string, instead of one row per product.

```
SELECT category,
       STRING_AGG(product_name, ', ') AS products
FROM products
GROUP BY category;
```

What it tests:

- STRING_AGG() combines multiple string values into a single delimited string.
- Like SUM() or COUNT(), it's an aggregate function typically used with GROUP BY.

Interview Tip:

- *"STRING_AGG() is an aggregate function that combines multiple row values into a single delimited string. We use it when a report needs one text list per group instead of multiple rows."*

22. FOREIGN KEY Violation on INSERT

Scenario: You try to insert an order for a store_id that doesn't exist in the stores table.

```
INSERT INTO orders (order_id, store_id, item, order_amount)
VALUES (5001, 'SFF-TOKYO', 'Chicken Burger', 250);
-- ERROR: insert or update on table "orders" violates foreign key constraint
```

What it tests:

- A FOREIGN KEY ensures every value in the child table exists in the parent table.
- If the referenced value doesn't exist, the database rejects the INSERT and no row is created.

Interview Tip:

- *"A FOREIGN KEY enforces referential integrity. If the parent key doesn't exist, the database rejects the INSERT and prevents orphan records from being created."*

23. Percentage of Total (Window Function)

Scenario: Finance wants each store's total revenue and what percentage of overall company revenue it contributes.

```
WITH store_revenue AS (
  SELECT store_id, SUM(order_amount) AS total_revenue
  FROM orders
  GROUP BY store_id
```

```

)
SELECT
    store_id,
    total_revenue,
    ROUND(total_revenue * 100.0 / SUM(total_revenue) OVER (), 2) AS
pct_of_total_revenue
FROM store_revenue;

```

What it tests:

- The CTE calculates each store's total revenue.
- SUM() OVER () calculates the overall company revenue while keeping every store row.
- Dividing each store's revenue by the overall revenue gives its percentage contribution.

Interview Tip:

- *"First calculate each store's revenue using a CTE, then use SUM() OVER () to get the overall revenue alongside every store row — this avoids a separate query or self-join."*

24. Indexes

Scenario: A query searching orders by customer_id takes several seconds on a table with millions of rows. After creating an index, the same query runs in milliseconds.

```

CREATE INDEX idx_orders_customer_id ON orders(customer_id);

SELECT * FROM orders WHERE customer_id = 4521;

```

What it tests:

- An index helps the database find matching rows faster without scanning the entire table.
- Indexes improve WHERE, JOIN, and ORDER BY performance, but add overhead to INSERT, UPDATE, and DELETE.

Interview Tip:

- *"An index is a lookup structure that speeds up data retrieval. We create indexes on columns that are frequently filtered, joined, or sorted, knowing that faster reads come with a small write overhead."*

25. Views

Scenario: You frequently need completed order details with customer and store information. Instead of repeating the same JOIN, the query is saved as a view.

```

CREATE VIEW completed_orders_summary AS
SELECT o.order_id, c.customer_id, s.store_name, o.order_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id

```

```
JOIN stores s ON o.store_id = s.store_id
WHERE o.status = 'Completed';

SELECT * FROM completed_orders_summary;
```

What it tests:

- A VIEW is a saved SQL query that can be queried like a table.
- It simplifies complex queries, avoids repeating SQL, and can control which data users can access.

Interview Tip:

- *"A view is a saved query, not a separate table. We use views to simplify complex SQL, improve reusability, and provide controlled access to data."*

26. Normalization

Scenario: Customer details are stored in every order, causing duplicate data and inconsistent updates. The database is redesigned by separating customers into their own table.

```
-- Before
-- orders(order_id, customer_name, customer_address, item, order_amount)

-- After
-- customers(customer_id, customer_name, customer_address)
-- orders(order_id, customer_id, item, order_amount)
```

What it tests:

- Normalization organizes data to reduce duplication and improve consistency.
- 1NF: atomic values (no repeating groups).
- 2NF: removes partial dependency.
- 3NF: removes transitive dependency.

Interview Tip:

- *"Normalization reduces data redundancy by storing each piece of information only once. 1NF ensures atomic values, 2NF removes partial dependency, and 3NF removes transitive dependency."*

27. DELETE/UPDATE Without WHERE

Scenario: You want to delete one cancelled order but accidentally forget the WHERE clause.

```
-- DELETE without WHERE - removes ALL rows
DELETE FROM orders;
```



```
-- DELETE with WHERE — removes only the intended row
DELETE FROM orders WHERE order_id = 5001;

-- Same risk with UPDATE
UPDATE orders SET status = 'Cancelled';           -- updates ALL rows
UPDATE orders SET status = 'Cancelled' WHERE order_id = 5002; -- updates only the
intended row
```

What it tests:

- DELETE without WHERE removes every row from the table.
- UPDATE without WHERE modifies every row.
- Neither asks for confirmation — it executes immediately.

Interview Tip:

- *"Without a WHERE clause, DELETE affects every row and UPDATE modifies every row. In practice, I first run a SELECT with the same WHERE condition to verify the affected rows, and use a transaction so I can ROLLBACK if needed."*

28. DELETE vs TRUNCATE vs DROP

Scenario: The task is to remove data from the database — which command should you use?

```
-- Remove specific rows
DELETE FROM orders WHERE status = 'Cancelled';

-- Remove ALL rows, keep the table
TRUNCATE TABLE orders;

-- Remove the table itself
DROP TABLE orders;
```

What it tests:

- DELETE removes selected rows and can use WHERE.
- TRUNCATE removes all rows but keeps the table structure.
- DROP removes the entire table, including its structure and data.

Interview Tip:

- *"DELETE removes rows, TRUNCATE removes all rows while keeping the table, and DROP removes the table itself. I think of it as a scope ladder — some rows, all rows, or the table is gone entirely."*

29. Replacing NULLs — COALESCE vs CASE

Scenario: The customers table has some rows where phone is missing. Display "NA" wherever the value is NULL — using two different approaches.

```
-- Method 1: COALESCE
SELECT customer_id,
       COALESCE(phone, 'NA') AS phone
FROM customers;

-- Method 2: CASE
SELECT customer_id,
       CASE WHEN phone IS NULL THEN 'NA' ELSE phone END AS phone
FROM customers;
```

What it tests:

- Both produce the same result here.
- COALESCE(a, b) returns the first non-NULL value from its list — purpose-built for NULL substitution, more concise.
- IS NULL does the same thing through explicit conditional logic, and is more flexible when the replacement logic isn't a simple "is this NULL" check.

Interview Tip:

- *"For a straight NULL-to-default substitution, COALESCE is the cleaner choice — it's shorter and it's what the function is designed for. I'd reach for CASE instead when the replacement logic is more complex than just checking for NULL, since COALESCE can only ever check for NULL, not arbitrary conditions."*

30. Finding Duplicates — Two Ways

Scenario: Find customer_name values that appear more than once in the customers table — duplicate customer names entered separately.

```
-- Method 1: GROUP BY + HAVING
SELECT customer_name, COUNT(*) AS occurrences
FROM customers
GROUP BY customer_name
HAVING COUNT(*) > 1;

-- Method 2: Window function (ROW_NUMBER) — also lets you see/keep specific duplicate
rows
SELECT *
FROM (
    SELECT customer_id, customer_name,
           ROW_NUMBER() OVER (PARTITION BY customer_name ORDER BY customer_id) AS rn
    FROM customers
) t
WHERE rn > 1;
```

What it tests:

- GROUP BY + HAVING COUNT(*) > 1 tells you which customer_name values are duplicated, but only returns the name and count — not the actual duplicate customer_id rows behind them.
- ROW_NUMBER() OVER (PARTITION BY customer_name) numbers each occurrence within its group, so filtering WHERE rn > 1 gives you every duplicate row itself.

Interview Tip:

- *"GROUP BY + HAVING answers 'which names are duplicated.' ROW_NUMBER PARTITION BY answers 'give me the actual duplicate customer rows' — which is what I'd need if the next step is deciding which record to keep and which to merge or delete, not just reporting a count."*

SQL Interview Guide with Scenarios.

LinkedIn: [GRASSP Acad](#)

Instagram: [@grasspacad](#)

SQL Free Youtube Fasttrack Series: [SQL Fast Track Series](#)

SQL Udemy course: [Complete SQL for Data Analysis - Scenario Based Learning](#)